



LAB # 12

SINGLE LAYERED FEED FORWARD NEURAL NETWORK (PERCEPTRON)

Objectives:

- To implement perceptron in Python

Hardware/Software Required:

Hardware: Desktop/ Notebook Computer

Software Tool: Python 3.6/3.7

Introduction:

Perceptron:

The perceptron is a supervised classification algorithm with a linear decision boundary, i.e. a classification algorithm that makes its predictions based on a linear predictor function combining a set of weights with the feature vector. The algorithm allows for online learning, in that it processes elements in the training set one at a time. The perceptron algorithm dates back to the late 1950s. Its first implementation, in custom hardware, was one of the first artificial neural networks to be produced.

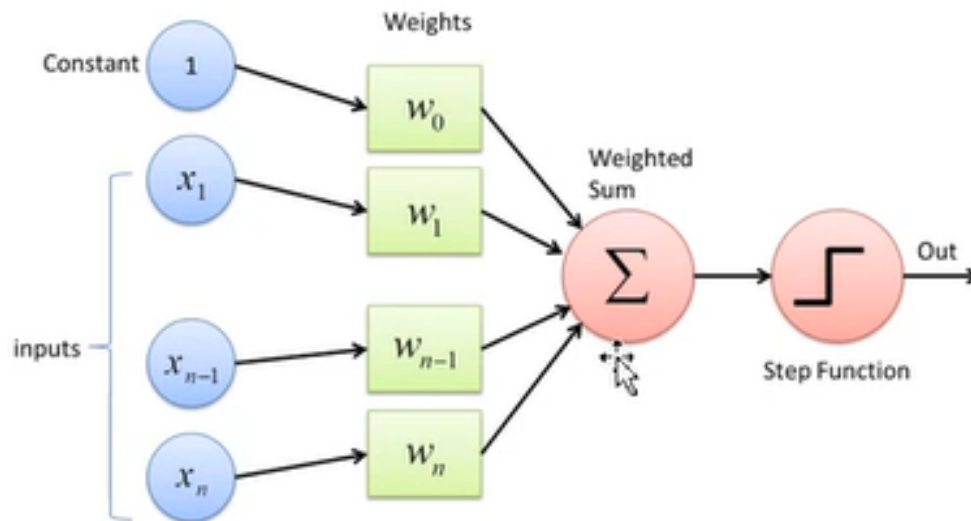
Perceptron maps a feature vector x to an output binary value (0 or 1) using the following relation:

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > t \\ 0 & \text{otherwise} \end{cases}$$

where w is a vector of real-valued weights, $w \cdot x$ is the dot product i.e. $\sum_{i=0}^m w_i x_i$, where m is the number of inputs to the perceptron, b is the bias and t is the activation threshold. The bias shifts the decision boundary away from the origin and does not depend on any input value. If the sum of dot product and the biasing factor is greater than the activation threshold, the output of a perceptron is fired otherwise it is not.

Weights can be updated using following formula:

$$New_{weight} = old_{weight} + 0.1 \times (target - output) \times input$$



x0	x1	x2	class
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Weights can be updated using following formula:

$$New_{weight} = old_{weight} + 0.1 \times (target - output) \times input$$

The pseudo code for the perceptron is given below:

Algorithm: Perceptron

Split the dataset into training and testing

Initialize weights and activation threshold 't' with 0

For iter = num iterations:

 For each sample 'x' in training



Split the class 'c' and features 'f' of x from each other

Compute dot product of weights and f add them up and store it in 'o'

if $o \geq t$

fire output i.e. $o = 1$

else

$o = 0$;

compute i.e. $\text{error} = \text{learningRate} * (c - o)$

for $j = \text{number of features}$

update weights i.e. $\text{weights}[j] = \text{weights}[j] + f[j] * \text{error}$

predict output for training data using the weights

Compute accuracy by analyzing predicted Outputs and actual Outputs

Lab Task:

- 1) Implement perceptron for the dataset given below with threshold=0.5 and learning rate 0.1. Update weights till all data points are correctly classified.

x0	x1	x2	class
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Bonus Task:

- 1) Use the given SONAR dataset and classify it using perceptron:
- First create a Python script and load 'SONAR.csv' file.
 - Identify features and classes from the loaded dataset.



- c) Shuffle the dataset and Split it in such way that 80% of dataset is used for training and 20% for testing.
- d) Implement the perceptron using the above algorithm and use training dataset to classify each of the sample within testing dataset. Use learning rate of 0.1, Threshold of 0.5 and 0 biasing factor with 0 initial weights.
- e) Repeat the above process until the training accuracy of 75% or above is not achieved.
- f) Use those weights to predict the classes for testing data and find the accuracy of prediction.
- g) Plot the confusion matrix for the test data classification.

Conclusion:

Write the conclusion about this lab

NOTE: A lab journal is expected to be submitted for this lab.